

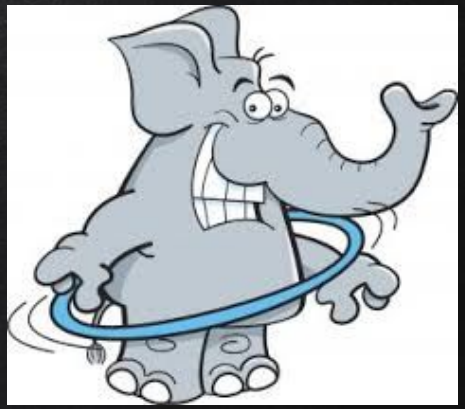
Teórica 3

Engenharia de Software

Processos - Agile

PORQUE É QUE O COMPUTADOR ESTÁ SEMPRE A CONGELAR?

Sumário



Processos Agile

Scrum

DevOps

Processos Agile: Scrum



Scrum

Problema:

Os processos Agile originais eram opacos para os gestores, porque:

- as equipas organizavam-se autonomamente
- não produziam documentação
- planeavam o desenvolvimento em ciclos muitos curtos

- Processos Agile têm de ser geridos de forma a optimizar a utilização do tempo e recursos
- Scrum: método agile para organizar métodos agile

O Processo Scrum: Papéis

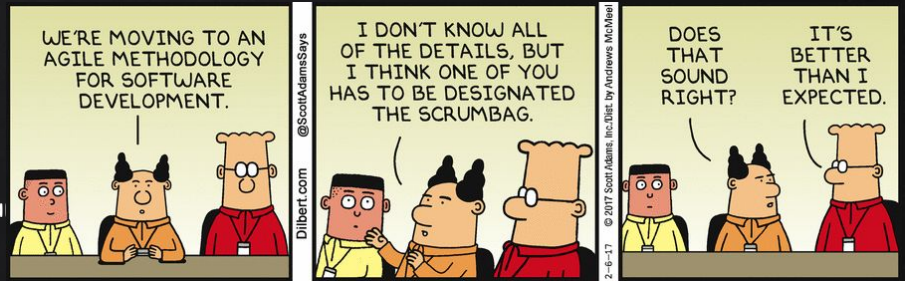
Scrum Team de desenvolvimento– ST (até 10 pessoas – Scrum Guide oficial de novembro 2020, Ken Schwaber e Jeff Sutherland):

- *Product Owner* – PO (identifica requisitos)
- *ScrumMaster* – SM (responsável por garantir o processo Scrum, evitando interferência externa)
- *Developers* – (competências técnicas necessárias para criar o Incremento)

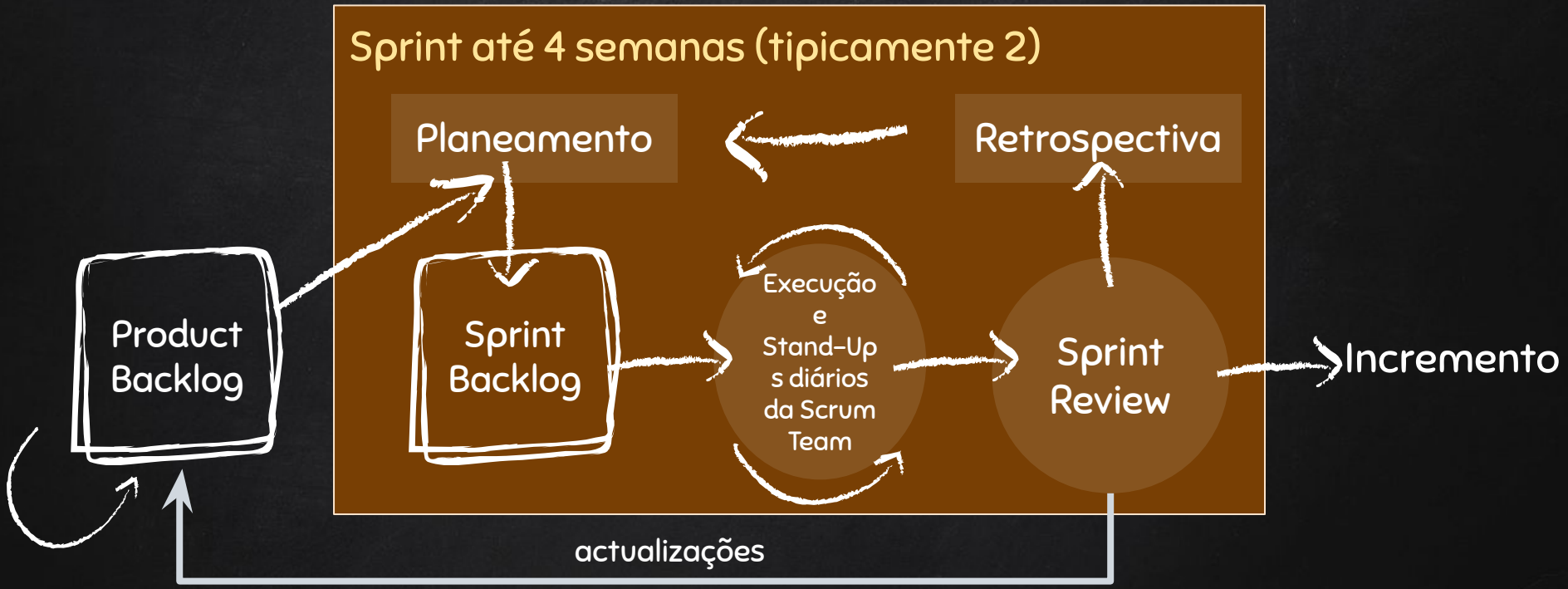
As tarefas são escolhidas pelos membros da equipa individualmente (organização autónoma)

Todos os membros da equipa devem ser capazes de fazer tudo (multi-functional)

O código é propriedade colectiva.



O SPRINT



Desenvolvimento da iteração em tempo limitado para produzir um produto funcional (entregável)

Duraçao do sprint

- **2 semanas (o padrão mais popular):** Equilibra tempo suficiente para desenvolver trabalho com valor real e um ciclo de feedback rápido com os utilizadores e stakeholders.
- **1 semana:** Usado em startups, ambientes de inovação rápida ou equipas muito maduras em DevOps/CI-CD que lançam funcionalidades em produção diariamente.
- **3 a 4 semanas:** Comum em indústrias com restrições regulamentares mais pesadas, sistemas legados complexos ou dependências de hardware.

Epics

Epics são *User Stories* (US) com um propósito global que ainda não foi refinado

- Demasiado grandes para serem concluídas num único *sprint*
- Devem ser divididas em US mais pequenas
- A equipa nuclear desenvolve-as com base na visão do projecto.

Definições dos Sprints

Product Backlog (lista de actividades "*to do*")

Daily Scrum (reunião diária, presencial, de toda a equipa, onde o trabalho é revisto e priorizado)

Velocidade (estima quanto esforço no *product backlog* uma equipa consegue cobrir num *sprint*)

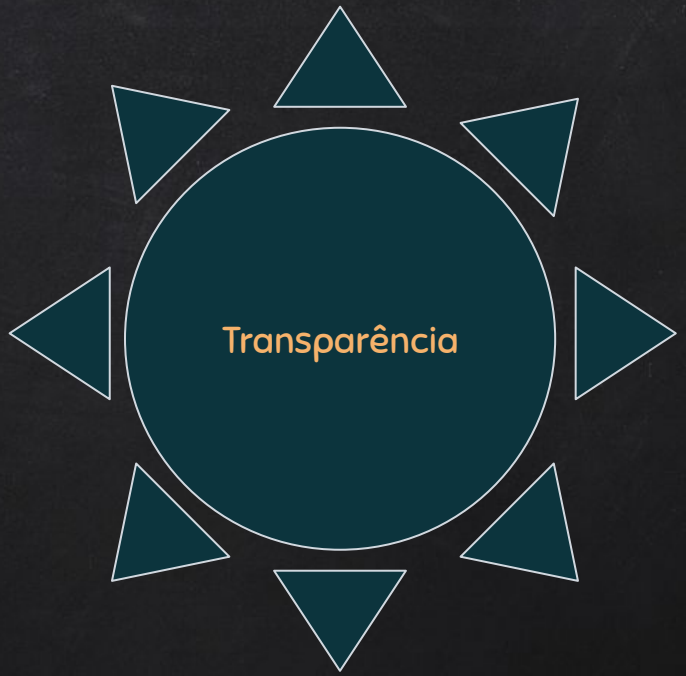
- Métricas:
  - Velocidade: número de US entregues por *Sprint*
  - Valor: Mede o valor comercial das US entregues
  - Número de *User Stories*: número de US entregues

Artefactos

- Visão do Projecto
- Product Backlog Priorizado
- Calendário de Planeamento Entregas

Reuniões

- Reunião de Sprint Review
- Standup Meeting Diária



Indicadores de processo

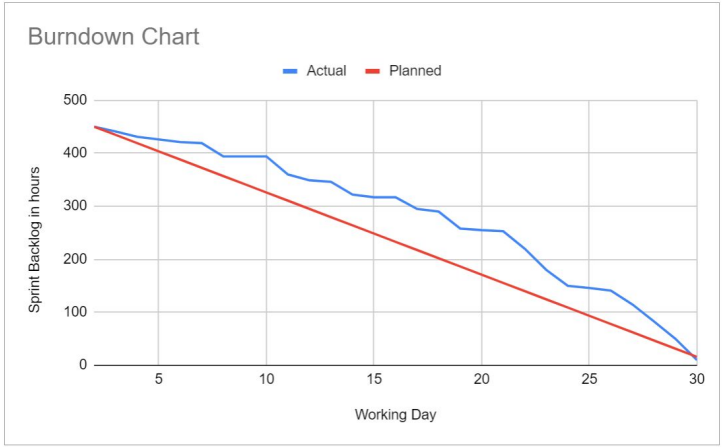
- Burndown Chart
- Scrum Board

Burndown chart

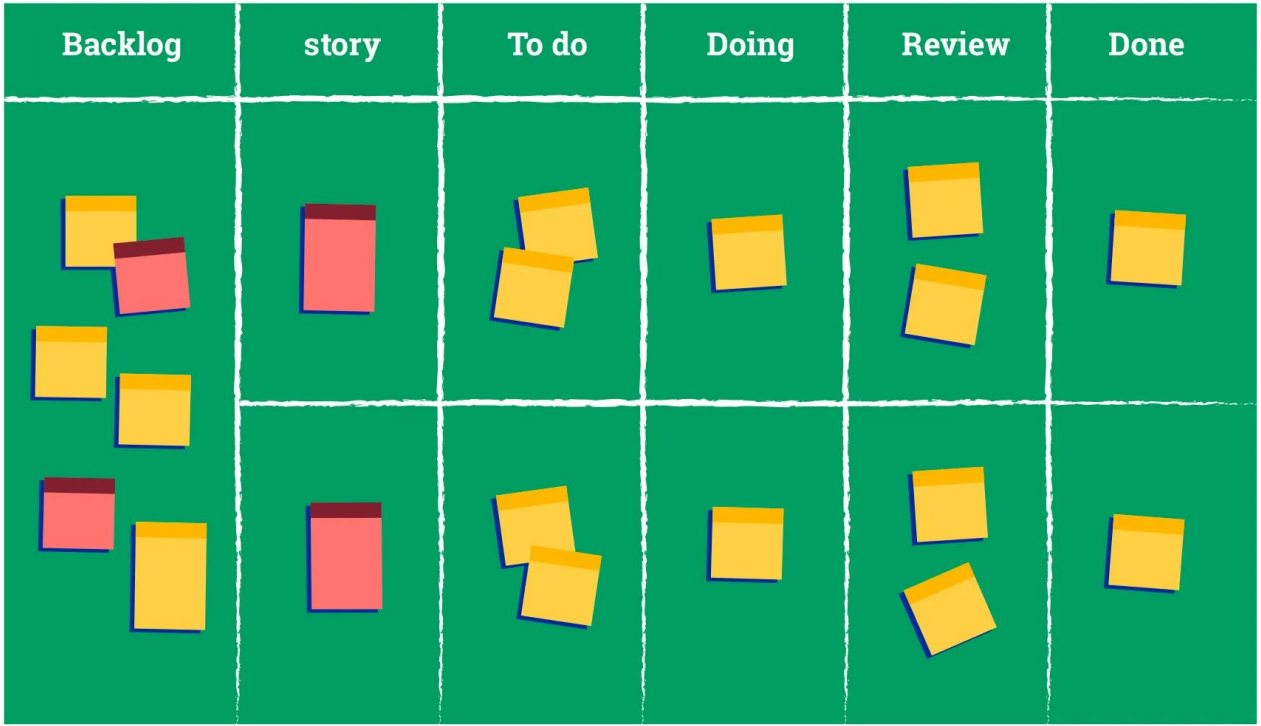
THE SPRINT BURNDOWN CHART

Team code			**Current Day**		
Team Oracle		hrs	**Average Work / Day**		hrs

	Work in progress	
Day	**Done**	**Accum**
1	0	0
2	9	9
3	10	19
4	5	24
5	5	29
6	2	31
7	25	56
8	0	56
9	0	56
10	34	90
11	11	101
12	3	104
13	24	128
14	5	133
15	0	133
16	22	155
17	5	160
18	32	192
19	3	195
20	2	197
21	33	230
22	40	270
23	30	300
24	4	304
25	5	309
26	26	335
27	32	367
28	33	400
29	40	440
30	10	450



Scrum board



Scrum – Papéis

Papéis Centrais

- Equipa Principal Scrum (*Core Team*):
  - Product Owner (PO)
  - Scrum Master (SM)
  - Scrum Team (ST)

Nenhum destes papéis tem autoridade sobre os outros!

Product Owner – A voz do Cliente

- Representa as partes interessadas (*stakeholders*)
- Responsável por garantir que a Scrum Team gera valor para o cliente
- Olhos e ouvidos do mundo exterior
- Ponto de contacto entre o cliente e os outros *stakeholders*
- Gasta o seu tempo conhecendo o produto (clarifica o que se espera do produto, datas de entrega, …)
- Decide as funcionalidades que devem ser atingidas durante o *sprint*
- Descreve os requisitos em US em conjunto com a *scrum team* (ou supervisiona a sua escrita pela equipa)

Responsabilidades do Product Owner (1/2)

Process	Responsibility
P01 – Create project Vision	Define the project vision.
P02 – Identify Scrum Master & Stakeholders	Choose the Scrum Master for the project. Identify the interest parties/stakeholders (e.g. users, sponsors,...).
P03 – Form the Scrum Team	Help to select the ST elements. Help creating the collaboration and team building plans with the help of the SM.
P04 – Develop Epics	Create the Epics and the Personas.
P05 – Create Prioritized Product	Prioritize the Product Backlog. Define the acceptance criteria.
P06 – Conduct Release Planning	Devise the Release Planning. Help to determines the Sprint duration.
P07 – Create User Stories	Help creating the US and define acceptance criteria.
P08 – Approve, Estimate and Commit US	Approve the US. Facilitates the organization of the ST meetings and make compromise with the US.
P09 – Create Tasks	Explain the US to the ST when they are building the task list.

Responsabilidades do Product Owner (2/2)

Process	Responsibility
P10 – Estimate Tasks	Give guidelines and answer to doubts when estimating tasks.
P11 – Create Sprint Backlog	Help ST to create the sprint backlog.
P12 – Create Deliverables	Clarify the business requirements to the ST.
P14 – Groom Prioritized Product Backlog	Revise and update the Prioritized Product Backlog.
P16 – Demonstrate and Validate Sprint	Accept/Reject deliverables. Give feedback to SM and ST. Update Release planning and prioritized backlog.
P18 – Ship Deliverable	Helps to deliver product increments.
P19 – Retrospect Project	Participate in the Project Retrospective Meeting.

Scrum Master

Liderança ao serviço da equipa

- Lider Informal preocupado com a estrutura interna Scrum
- Garante que o método é correctamente utilizado
- Facilitador, influenciador, mentor
- Garante que a ST tem todo o suporte necessário
- Isola a equipa de interferências externas

Responsabilidades do Scrum Master (1/2)

Process	Responsibility
P02 – Identify Scrum Master & Stakeholders	Helps identifying the interested parts.
P03 – Form the Scrum Team	Facilitates the selection of ST elements. Facilitates the creation of the collaboration and team building plan.Ensures that the backup resources are available to be put in action.
P04 – Develop Epics	Facilitates the creation of Epics and Personas.
P05 – Create Prioritized Product	Helps the PO to create the Prioritized Product Backlog.
P06 – Conduct Release Planning	Coordinates the Release Planning. Determines the Sprint duration together with PO.
P07 – Create User Stories	Supports the ST creating the US and acceptance criteria.
P08 – Approve, Estimate and Commit US	Facilitates ST meetings to estimate and create US.
P09 – Create Tasks	Supports the ST creating the tasks list to be done during the next Sprint.

Responsabilidades do Scrum Master (2/2)

Process	Responsibility
P10 – Estimate Tasks	Supports the ST in the process of estimating the task list for the next Sprint.
P11 – Create Sprint Backlog	Supports the ST to update the Sprint Backlog and the Sprint burndown chart.
P12 – Create Deliverables	Supports the ST handling with the deliverables.
P13 – Conduct Daily Standup Meetings	Ensures that the Scrum board and impediment log are updated.
P14 – Groom Prioritized Product Backlog	Facilitates meetings for revising the Prioritized Product Backlog.
P15 – Convene Scrum of Scrums	Ensures that the questions affecting the ST are solved.
P16 – Demonstrate and Validate Sprint	Facilitates the presentation of the results of the Sprint to the PO to be approved(or not).
P17 – Retrospect Sprint	Ensures that there is the ideal setting so that the ST has success in its work.
P19 – Retrospect Project	Represent the Scrum Core Team during the presentation and register of learnt lessons.

Scrum Team

- Conjunto de elementos multidisciplinares e heterogéneos
- Responsável por compreender os requisitos do negócio apresentados pelo PO, por estimar as US e produzir entregáveis
- Responsável por produzir o *output* correcto
- Equipa pequena até 10 pessoas
- Princípio dos Mosqueteiros: Um por Todos e Todos por Um!
- Organização e sustentabilidade autónomas
- Todos os membros estão envolvidos em todas as decisões relacionadas com os entregáveis

Scrum Team

- Promotes face-to-face meetings
- Focused on Value (value teamwork, continuous improvement of their performance and its outputs, working conditions)
- Promotes iterative product Delivery (for adjustments and improvements)

Responsabilidades da Scrum Team (1/2)

Process	Responsibility
P03 – Form the Scrum Team	Gives inputs for creating the team building collaboration plan.
P04 – Develop Epics	Ensures that there is a clear understanding of Epics and Personas.
P05 – Create Prioritized Product	Understand the Prioritized Product Backlog.
P06 – Conduct Release Planning	Accept the Sprint duration. Clarify the requests of new product and/or changes of the existing products in the Prioritized Product Backlog.
P07 – Create User Stories	Give inputs to the PO when creating the US.
P08 – Approve, Estimate and Commit US	Estimate the US approved by the PO.
P09 – Create Tasks	Develop the Task List based on the approved US.

Responsabilidades da Scrum Team (2/2)

Process	Responsibility
P10 – Estimate Tasks	Estimate the identified tasks and update the Task List.
P11 – Create Sprint Backlog	Develop the Sprint Backlog and the Sprint burndown chart.
P12 – Create Deliverables	Execute the task list, according to their dependencies and acceptance criteria. Identify obstacles and risks, and implement risk mitigation. Update the impediment log.
P13 – Conduct Daily Standup Meetings	Update Sprint Burndown Chart, Scrum Board and impediment list. Discuss problems and solutions to motivate the team. Identify risks and submit changes requests.
P14 – Groom Prioritized Product Backlog	Participate in meetings for revising the Prioritized Product Backlog
P15 – Convene Scrum of Scrums	Give input to the SM for the SoS meeting.
P16 – Demonstrate and Validate Sprint	Present the produced product achievements to the PO for approval.
P17 – Retrospect Sprint	Identify improvement opportunities for the next Sprints.
P19 – Retrospect Project	Participate in the Project Retrospective Meeting.

Os benefícios do Scrum podem ser mais difíceis de atingir quando:

- Os membros da equipa estão geograficamente dispersos ou a tempo parcial
- Os membros têm valências demasiado especializadas
- Os produtos têm muitas dependências externas
- Produtos que já estão maduros ou *legacy* ou com controlo de qualidade regulado

CONTINUOUS DELIVERY E DEVOPS

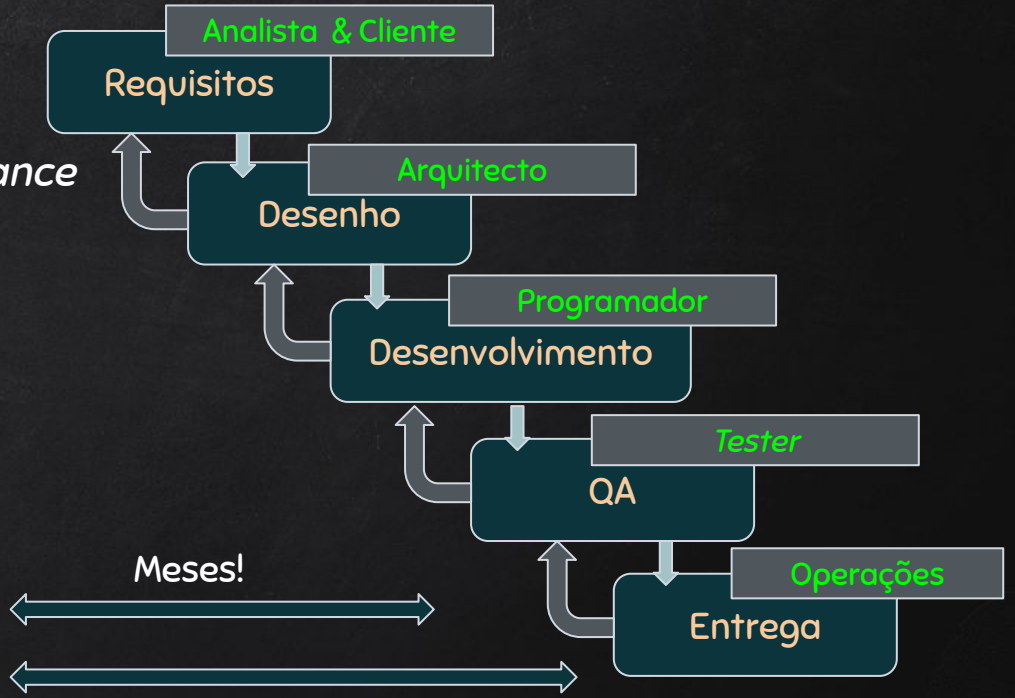


Modelo Cascata



Problemas

- Longo ciclo de *feedback* de *Quality Assurance*
- Falta de colaboração
- Difícil reação à mudança



Projectos Agile

- *Deployments* manuais
- Gestão de configuração insuficiente
- *Deployments* pouco frequentes e sujeitos a erro



Entrega – é o processo de fazer *deploy* de aplicações para ambientes de produção de uma forma consistente e confiável. Esta fase também inclui fazer o *deploy* e a configuração da infraestrutura de suporte aos ambientes de produção.

Operações – Esta fase envolve manter, monitorizar e fazer *troubleshooting* de aplicações em ambientes de produção.

Continuous Delivery

Um conjunto de práticas e princípios que visam construir, testar e lançar software com mais rapidez e frequência.

Lembrem-se de um dos doze princípios do Manifesto Agile:

"A nossa maior prioridade é, desde as primeiras etapas do projecto, satisfazer o cliente através da entrega rápida e contínua de software com valor."



QUALIDADE

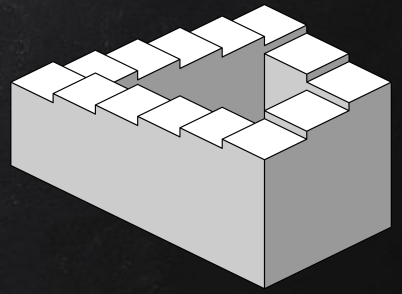
OBJECTIVOS

CICLOS DE TEMPO



Quanto tempo demora a organização a implementar uma mudança na produção que envolva apenas uma linha de código?

Continuous Delivery



- Continuous Integration (CI)
- Continuous Deployment (CD)
- Continuous Experimentation (CE)
- DevOps

Continuous delivery com Deployment Automatizado

É a práctica de integrar, enviar código para um repositório central (como o git), fazer *build*, testá-lo automatica e frequentemente (e.g., várias vezes por dia, diariamente ou semanalmente) e ter uma versão pronta e testada para ser *deployed*.

Pipeline de deployment Automatizado

Entrega automática, directa e regular ao utilizador final.

Um *pipeline* de *deployment* automático é uma sequência de *scripts* que é executado a cada alteração no código que é *committed* para o repositório:



O processo não avança se for detectado alguma problema num dos passos!

Continuous Integration





Verifica se o código escrito por diferentes programadores está integrado, dando feedback aos programadores.

Assim que o novas alterações são *pushed*, as ferramentas de CI compilam o código do repositório, correm testes unitários e verificam se as alterações não introduziram defeitos.

Ferramentas que implementam *Continuous Integration*:

Jenkins, cruise control, travis-CI, gitlab, github (com gestão de configurações)...

Automated Acceptance Testing





É escrito um conjunto de testes, em conjunto com o cliente, no início do processo.

Estes testes representam requisitos funcionais vistos da perspectiva do negócio. São escritos na forma de histórias, ou exemplos, pelos clientes e pelos programadores, para que concordem na forma como o software deve funcionar.

Funcionam como um controlo de qualidade para decidir se o produto está ou não pronto para ser lançado.

Gestão de Configuração





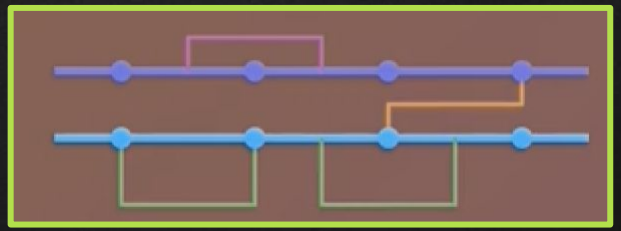
Responsável por monitorizar e controlar alterações (versionamento) no software e no seu ambiente. Envolve:

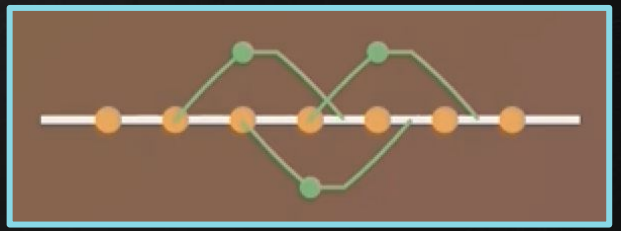
- preparar e instalar as ferramentas necessárias
- escalar o número de instâncias do serviço e a sua distribuição
- inventariar a infraestrutura
- e todas as tarefas relacionadas com o *deploy* da aplicação

Estratégias de Controlo de Versões (mesma ferramenta):

GitFlow *vs*

Trunk-based





Git flow vs Desenvolvimento Trunk-Based

Git flow

- *Branches* múltiplos e duradouros para funcionalidades, *releases* e correções

Pros:

- Controlo sobre a qualidade do código e o processo de lançamento
- Lançamentos em ciclos planeados & múltiplas versões
- Suporte para funcionalidades complexas e compatibilidade com versões anteriores

Cons:

- Gestão de conflitos entre versões
- Complexidade no processo de desenvolvimento
- Menos alinhado com CI/CD

Desenvolvimento Trunk-based

- Um único *branch* para todas as alterações ao código com *commits* pequenos e frequentes

Pros:

- Continuous Integration, ciclos de *feedback* mais rápidos, *deploy* mais rápido
- Menos conflitos
- Apropriado para equipas Agile & DevOps

Cons:

- Requer teste automatizado robusto e um *pipeline* CI/CD forte
- É desafiante sem uma cultura CI/CD forte
- Requer gestão cuidadosa de *feature flags* and revisões de código para manter a estabilidade

Continuous deployment (na prática)

- Os testes automáticos devem ser suportados por uma bateria de testes forte (para aumentar a confiança)

- A decisão de fazer *deploy* deve ser automática mas poderá haver um "botão de *deploy*" (que poderá ser restrito a algumas pessoas responsáveis)

- Na prática, alguns contextos implicam que, não só a equipa pode decidir prosseguir com o *deploy* mas também o cliente precisa de aceitar ou poderão existir restrições legais ou regulamentares. Por exemplo:
  - As apps do Android têm de ser aprovadas
  - As actualizações de serviços como o Facebook são realizadas sem que o cliente sequer se dê conta de que está a usar uma nova versão

Continuous Experimentation

Correr experiências e comparar diferentes versões do software, de forma contínua, no ambiente de produção.

É possível fazer o *deploy* de diferentes versões do software para perceber se algumas características são preferíveis.

Continuous experimentation



A ideia principal é estar a fazer experiências contínuas com os utilizadores.

Abordagem "Medicamentos" vs "placebo"

Características do Teste: versão/funcionalidade A (estável) comparada com a funcionalidade versão/A' (nova)

- Canary release (ter uma nova versão: a versão corrente está estável e 5% usam a nova versão) se houver erros, só 5% dos utilizadores é que são afectados
- A/B-testing (funcionalidade estável A e funcionalidade B usadas por 50% dos utilizadores, cada) Se tivermos uma indicação de as coisas estarem estáveis
- Dark launch (100% estão a utilizar a versão estável mas as interacções/acções são observadas, guardadas e testadas na nova versão, em paralelo)

Continuous experiment – Algumas considerações

Como se implementa?

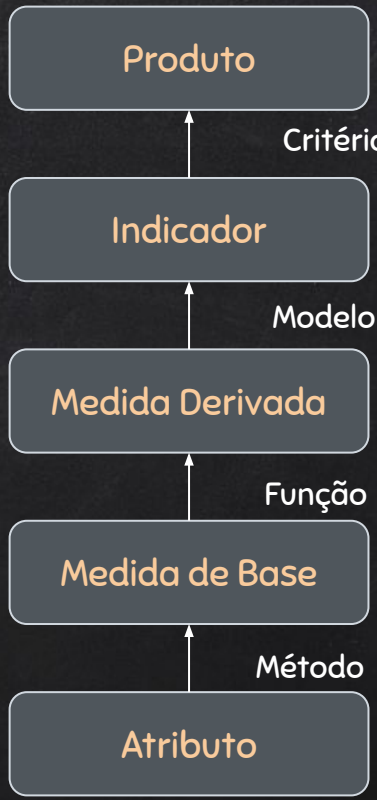
- Instrumentando o código com *feature toggles* (interruptor on/off) que ligam ou desligam a funcionalidade (instrução if)
- Pode ser feito ao nível da rede (e.g. baseado em IP, certos utilizadores em certos servidores)
- Medindo (pode usar bibliotecas para medir) o que possa correr mal (e.g. contar falhas). Usar indicadores para escolher a melhor versão
- Considerações científicas relativamente a factores Humanos (e.g. conhecimento sobre perfis de utilizador e amostragens, etc...)

Medições

- Para além de alimentarem a experimentação contínua, as medições são importantes para melhorar a comunicação, tomada de decisões, etc.
- Produto (normas internacionais SQuaRE):
  - ISO/IEC 25020:2019 – Modelo de enquadramento de medição da Qualidade
  - ISO/IEC 25021:2012 – Elementos de medição da qualidade
  - ISO/IEC 25022:2016 – Medição da qualidade em uso
  - ISO/IEC 25023:2016 – Medição da qualidade do sistema e do produto de software
  - ISO/IEC 25024:2015 – Medição da qualidade dos dados
- Processo (normas internacionais):
  - ISO/IEC/IEEE 12207:2017 – Engenharia de Sistemas e de Software – Processos de ciclo de vida de Software
  - ISO/IEC/IEEE 15288:2023, Systems and software engineering – Processos de ciclo de vida do sistema
- Como medir (norma internacional):
  - ISO/IEC/IEEE 15939:2017 – Engenharia de Sistemas e de Software – Processo de Medição

Medições: Informações de Medições Necessárias

E.g. Quão *buggy* é o nosso software? Quão seguro é?



E.g. utilizar *dashboards*/ferramentas de visualização para detectar situações problemáticas

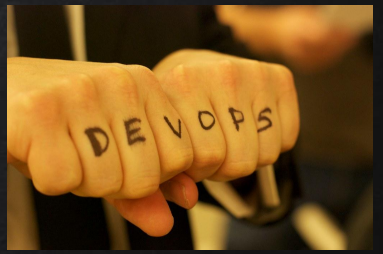
E.g. se tivermos um *bug* por cada mil linhas de código então estamos mal.

E.g. Número de *bugs* em comparação com o tamanho total do código?

E.g. quantas linhas de código? Número de classes? Número de funções? Número de *bugs*?

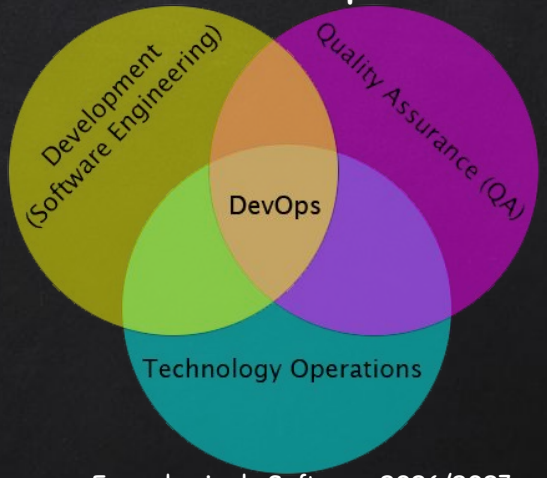
E.g. dimensão do código?

DevOps

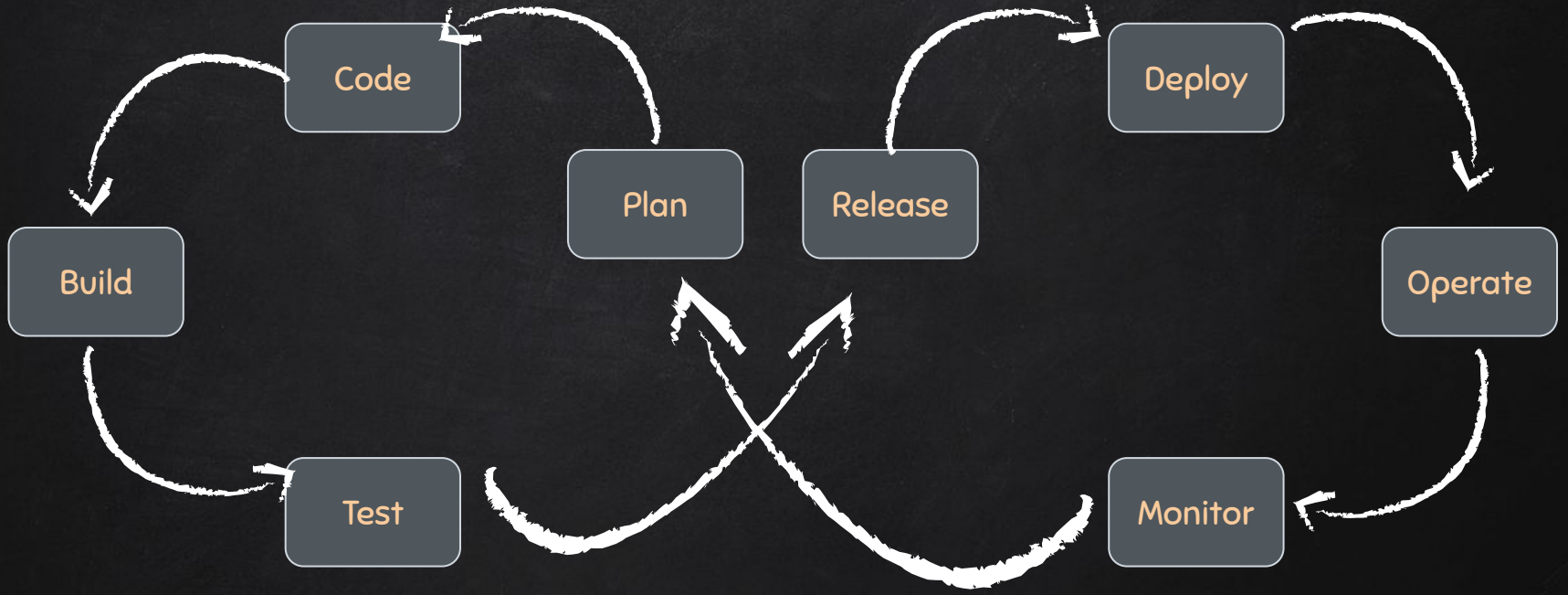


Combinação entre o desenvolvimento e a operação de tecnologias de informação (antigamente uma equipa desenvolvia e entregava o software a outra para o operar)

Em DevOps a equipa que desenvolve, também opera, corre, obtém *feedback* e evolui os conceitos.



DevOps



A equipa é responsável por tudo! Não existe a separação do Desenvolvimento, Garantia de Qualidade e Operações.

Desafios em DevOps

Desenho da Arquitectura

- Como posso tomar decisões sobre um produto que vai ser *deployed* regularmente?
- Como reverto alterações?
- Como consigo dados que me dêem informação valiosa?

Desafios em DevOps

Infraestrutura (que tecnologia deve ser usada?)

Teste

Gestão de Configurações

Os papéis mudam dentro da equipa (resistência à organização actual?)

Coordenação entre equipas

Descoberta (existe algum processo para encontrar novos requisitos?)

Feedback do cliente (como ter *feedback* contínuo do cliente se eles não aceitarem?)

Conclusões a retirar sobre os Modelos de Processo estudados:

Escolher um modelo de processos adequado é crucial:

- Quão bem compreendemos os requisitos?
- Qual é a esperança de vida do projecto?
- Qual é o nível de risco envolvido?
- Quais são as restrições de calendário?
- Qual é o nível de visibilidade necessário para a gestão externa?
- Qual é o nível de interacção com o cliente?
- A nossa equipa tem experiência com o processo?

PORQUE É QUE O COMPUTADOR ESTÁ SEMPRE A CONGELAR?

PORQUE DEIXOU AS JANELAS ABERTAS.



VEMO-NOS NA PRÓXIMA AULA



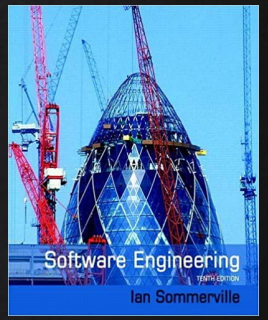
Referências

Software Engineering, by Ian Sommerville, Pearson, (9ª ou 10ª edição)

Ver capítulos 2 e 3

O Manifesto Agile: https://agilemanifesto.org/

Outras referências: Scrum A Gestão Ágil de Projectos, João Pinto, Christiane Tscharf, FCA editora



Quiz!

Qual das seguintes opções não caracteriza os valores do Manifesto Agile?

A. Indivíduos e interacções acima de processos e ferramentas.

B. Ferramentas sofisticadas e processos para aumentar a velocidade de desenvolvimento de software.

C. Software que funciona acima de documentação compreensiva.

D. Colaboração do cliente acima de negociação de contratos..

E. Resposta a mudança acima de seguir planos.

Quiz!

Qual das seguintes opções não caracteriza os valores do Manifesto Agile?

A. Indivíduos e interacções acima de processos e ferramentas.

B. Ferramentas sofisticadas e processos para aumentar a velocidade de desenvolvimento de software.

C. Software que funciona acima de documentação compreensiva.

D. Colaboração do cliente acima de negociação de contratos..

E. Resposta a mudança acima de seguir planos.

Quiz!

Quais das seguintes opções são exemplos de modelos de processo Agile?

A. Cascata, Incremental, Espiral
B. Cascata, XP, Espiral
C. Espiral, XP, Programação por Pares
D. Programação por Pares e Scrum
E. Espiral e Scrum

Quiz!

Quais das seguintes opções são exemplos de modelos de processo Agile?

A. Cascata, Incremental, Espiral
B. Cascata, XP, Espiral
C. Espiral, XP, Programação por Pares
D. Programação por Pares e Scrum
E. Espiral e Scrum

Quiz!

Test-Driven Development é tipicamente considerado em que modelo?

A. Espiral
B. Cascata
C. Prototipagem
D. RAD (Desenvolvimento Rápido de Aplicações)
E. XP

Quiz!

Test-Driven Development é tipicamente considerado em que modelo?

A. Espiral
B. Cascata
C. Prototipagem
D. RAD (Desenvolvimento Rápido de Aplicações)
E. XP

Quiz!

Qual das seguintes opções não é uma característica da abordagem Scrum?

A. A equipa organiza-se autonomamente.
B. Código como propriedade individual: cada programador é responsável pelo seu código.
C. A equipa é multifuncional
D. O *product owner* é responsável por identificar os requisitos
E. O *Scrum Master* é responsável por assegurar o processo Scrum evitando interferência externa.

Quiz!

Qual das seguintes opções não é uma característica da abordagem Scrum?

A. A equipa organiza-se autonomamente.
B. Código como propriedade individual: cada programador é responsável pelo seu código.
C. A equipa é multifuncional
D. O *product owner* é responsável por identificar os requisitos
E. O *Scrum Master* é responsável por assegurar o processo Scrum evitando interferência externa.

Quiz!

No Scrum (selecionar a resposta correcta):

A. Um *sprint* é uma lista de actividades "to do".
B. Um scrum diário é uma actividade diária onde os programadores se encontram e fazem programação por pares.
C. A velocidade estima quanto esforço no *backlog* é que uma equipa consegue cobrir num sprint.
D. Um epic é um sprint com muito sucesso.
E. O *product backlog* é uma lista de *user stories* entregues por sprint.

Quiz!

No Scrum (selecionar a resposta correcta):

A. Um *sprint* é uma lista de actividades "to do".
B. Um scrum diário é uma actividade diária onde os programadores se encontram e fazem programação por pares.
C. A velocidade estima quanto esforço no *backlog* é que uma equipa consegue cobrir num sprint.
D. Um epic é um sprint com muito sucesso.
E. O *product backlog* é uma lista de *user stories* entregues por sprint.

Quiz time

Qual dos seguintes não se encontra num scrum board?

A. O conjunto de tarefas deste sprint num *burndown chart*.
B. O conjunto de tarefas do sprint actual.
C. O conjunto de tarefas que estão actualmente em "to do".
D. O conjunto de tarefas que já estão completas no sprint actual.
E. O conjunto de tarefas que estão em progresso no sprint actual.

Quiz time

Qual dos seguintes não se encontra num scrum board?

A. O conjunto de tarefas deste sprint num *burndown chart*.
B. O conjunto de tarefas do sprint actual.
C. O conjunto de tarefas que estão actualmente em "to do".
D. O conjunto de tarefas que já estão completas no sprint actual.
E. O conjunto de tarefas que estão em progresso no sprint actual.